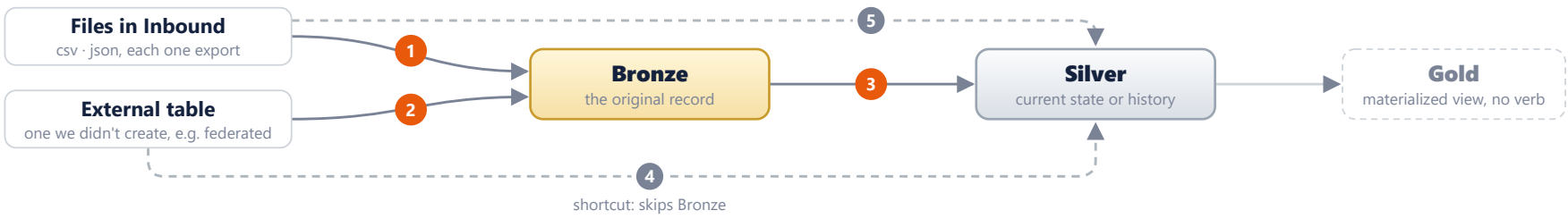
dbx-flame

Recommended source × strategy × verb combinations

One task is one hop. Pick the hop, then read the cards top to bottom: the first fits most datasets. A rejected combination fails before anything is written: most at Start, a re-read of a stamped table at its first read.

✓ Recommended · ✓ Possible · ! Valid, read the note · ✗ Rejected



Source `source.origin`

Where does the data come from?

Files in Inbound — CSV or JSON files in a volume. Each file is one export, and its file name gives the export date.

External table — A table that the framework did not write, for example a federated one. Each read gets the read time as its export date.

Stamped table — A table that the framework wrote, usually Bronze. Each row keeps the source and export date it had when it entered.

Strategy `source.increment_strategy`

Which rows does each run read?

`checkpoint` — A stream records the last position it read. The next run reads only the data that arrived after that position.

`watermark` — The run reads only the rows with an export date later than the newest export date in the target.

`+ anchor` — The watermark compares the date each row changed (`__ANCHOR_DT`), not the export date.

`full_read` — The run reads the full table. Each read is one complete snapshot.

`delta_read` — The run reads only the rows that changed after the last run. A date column in the source shows which rows changed.

Verb `output.verb`

How does the run write to the target?

`APPEND` — Adds the rows to the target. It never changes or removes a row.

`FULL` — Replaces the target with the newest export. An empty batch changes nothing.

`UPSERT` — Updates the row of each key, or adds a row for a new key. It keeps no history.

`SCD2` — Closes the current version of each changed key and adds the new version. It keeps the history.

`COMPLETE_DELTA` — Replays each export in date order, so each version of a row is in the history. It also applies a deletes feed.

`scope: delta` — The export holds only changes. A key that is not in the export stays as it is.

`scope: full` — The export holds the full dataset. The run closes each key that is not in the export.

1 Files → Bronze

Auto Loader, origin csv · json

✓ `checkpoint` → `APPEND`
Bronze keeps **every export exactly as it arrived**, stamped with its file and export date.

✓ `checkpoint` → `FULL` `UPSERT` `SCD2`
Works, but Bronze **stops being the original record**.

REJECTED

- ~~`watermark` `full_read` `delta_read`~~: Auto Loader only resumes from its checkpoint
- ~~`COMPLETE_DELTA`~~: replays snapshots from a table, not files

2 External table → Bronze

each read is stamped as one export

✓ `full_read` → `APPEND`
One **complete snapshot** per run, stamped with the read time. **Sees deletes.**

✓ `delta_read` → `APPEND`
Only rows changed since the last run, for a **table too large to read whole**. Needs `anchor_dt`; deletes are invisible.

✓ `full_read` → `FULL`
Current state only: past reads are not kept.

REJECTED

- ~~`delta_read → FULL`~~: would replace the table with only the changed rows
- ~~`checkpoint` `watermark`~~: no ~~`__EXPORT_DATE`~~ to follow

3 Bronze → Silver

a table we stamped, read export by export

✓ `checkpoint` → `FULL`
Only the latest matters. The newest export becomes the current state.

✓ `checkpoint` → `UPSERT`
Latest version per key (SCD1), no history. Deleted keys stay.

✓ `checkpoint` `watermark` → `SCD2`
History of changes, latest version per batch. Deleted keys stay.

✓ `watermark` → `COMPLETE_DELTA` `scope: delta`
Replays every export in order, whole or partial. Omitted keys stay current; only a deletes feed retires them.

✓ `watermark + anchor` → `COMPLETE_DELTA` `scope: delta`
Reads only rows whose `__ANCHOR_DT` moved: a **full feed resent unchanged costs nothing**.

! `watermark` → `COMPLETE_DELTA` `scope: full`
Absent = deleted, history kept. Only over a Bronze that holds **whole exports**: files or `full_read` + APPEND.

REJECTED

- ~~`anchor → scope: full`~~: reads only moved rows; the rest would be retired
- ~~`full_read` `delta_read`~~: would stamp every export again

4 External table → Silver

shortcut: prefer lanes 2 + 3

! `full_read` → `UPSERT` `SCD2` `COMPLETE_DELTA`
Skips Bronze, so **no original record is kept**. Land it in Bronze first.

! `delta_read` → `UPSERT` `SCD2` `COMPLETE_DELTA` `scope: delta`
Same, and **deletes never reach Silver**.

REJECTED

- ~~`delta_read → COMPLETE_DELTA scope: full`~~: each read is partial, every row it lacks would be retired
- ~~`delta_read`~~ + ~~`deletes_table`~~: land the deletes in Bronze with a ~~`full_read`~~ task

5 Files → Silver

shortcut: prefer lanes 1 + 3

! `checkpoint` → `FULL` `UPSERT`
Current state only, safe in a backlog: the newest export, or the newer row per key, wins. Skips Bronze, so **no original record is kept**.

! `checkpoint` → `SCD2`
Each batch keeps **only the latest version per key**: in a backlog, the exports in between never reach the history.

REJECTED

- ~~`watermark` `full_read` `delta_read`~~: Auto Loader only resumes from its checkpoint
- ~~`COMPLETE_DELTA`~~: replays exports from a table: land the files in Bronze first

Set `output.event_time.column` to a real change date. The keyed verbs use this column, not the row content, to find a change. If it is `__EXPORT_DATE`, each export adds a new version of every unchanged row in SCD2 and COMPLETE_DELTA (gh #51).

`anchor_dt` and `event_time` are two settings. `source.anchor_dt` selects the rows that the next run reads. `output.event_time` tells a keyed verb which version is newer. Both can name the same change date, for example `MODIFIED_AT` or its Bronze copy `__ANCHOR_DT`. They can also name different columns.

Full and delta feeds of one source are two tasks that write to the same Silver. The full task (`scope: full`) runs first, and the anchored delta task runs second. The task dependencies of the job set this order, not the framework.